Trường Đại học Nguyễn Tất Thành
Khoa Công Nghệ Thông Tin

KỸ THUẬT LẬP TRÌNH

TS. Võ Thị Hồng Thắm

CHƯƠNG IV.
KẾ THỪA

NỘI DUNG

1. Tái sử dụng mã nguồn trong việc phát triển ứng dụng
2. Thừa kế
3. Sub class và super class
4. Lớp trừu tượng – Phương thức trừu tượng

Resued :: Tái sử dụng

- Tồn tại nhiều loại đối tượng có các thuộc tính và hành vi tương tự hoặc liên quan đến nhau:
  - Person, Student, Manager, Lecture, …
  - Merchandise, Product, Clothes, …
- Xuất hiện nhu cầu sử dụng lại các mã nguồn đã viết
  - Sử dụng lại thông qua copy
  - Sử dụng lại thông qua quan hệ *has_a*
  - Sử dụng lại thông qua cơ chế "kế thừa"

Resued :: Tái sử dụng

- Copy mã nguồn
  - Tốn công, dễ nhầm lẫn, tốn kém dung lượng lưu trữ
  - Khó sửa lỗi do tồn tại nhiều phiên bản
  - Thiếu nhất quán
- Quan hệ *has_a*
  - Sử dụng lớp cũ như là thành phần của lớp mới
  - Sử dụng lại cài đặt với giao diện mới
    - Phải viết lại giao diện
    - Cứng nhắc, Thiếu linh hoạt, đôi khi không đáp ứng được yêu cầu truy xuất

Resued :: Tái sử dụng

- Minh họa **has_a**

```
class Person {
    private String name;
    private Date birthday;
    public String getName() { return name; }
...
}
class Employee {
    private Person me;
    private double salary;
    public String getName() { return me.getName(); }
...
}
```

Resued :: Tái sử dụng

- Minh họa **has_a**

```
class Manager {
   private Employee me;
   private Employee assistant;
   public setAssistant(Employee e) {...}
   ...
}
...
Manager junior = new Manager();
Manager senior = new Manager();
senior.setAssistant(junior);   //<== lỗi vì junior không phải là Employee
```

Inheritance :: Thừa kế

- Dựa trên quan hệ *is_a*
- Thừa hưởng lại các thuộc tính và phương thức đã có từ lớp cha (*super class*)
- Chi tiết hóa cho phù hợp với mục đích sử dụng tại lớp mới
  - Thêm các thuộc tính mới
  - Thêm hoặc hiệu chỉnh các phương thức đã có theo "*mong đợi*" của lớp mới

Inheritance :: Thừa kế

- Java cung cấp cơ chế phân cấp có thứ bậc giữa các class, gọi là cây thừa kế
- Class bậc trên gọi là lớp cha (*super class*), các class bậc dưới được gọi là các lớp con (*sub class*)
- Trong Java, 1 lớp chỉ có thể có 1 lớp cha duy nhất (*đơn thừa kế*), điều này thuận với tự nhiên, tránh bị xung đột.

Inheritance :: Thừa kế

Cú pháp:

```
[public | protected] class <subclass>
extends <superClass> {
        /*--- new features goes here */
}
```

- Sử dụng từ khóa **extends** để chỉ ra lớp cha (super class) muốn thừa kế

Example

```
class Employee extends Person {
    private double salary;
    public void setSalary(double sal) {
        this.salary = sal;
    }
}
```

```
Employee e = new Employee();

e.setName("John");
e.setSalary(3.0);
```

Một số đặc điểm cần lưu ý

- Lớp con được "*thừa hưởng*" các tài sản từ lớp cha (*field, method of super class*), tuy nhiên việc "*thừa hưởng*" này phải tuân theo một số đặc điểm như sau:
  - sub class luôn được thừa hưởng các thành phần thuộc loại **public** hoặc **protected** của super class
  - sub class chỉ được thừa hưởng các thành phần mặc định {default} của super class nếu cả sub class và super class đều cùng thuộc 1 package
  - Mặc dù được thừa hưởng mọi thức từ super class, tuy nhiên những thành phần được khai báo là **private** trong super class thì sub class không được phép truy xuất
  - constructor không cho phép thừa kế

Bạn đã hiểu?

```
package poly.ho;
public class NhanVien{
    public String hoTen;
    protected double luong;
    public NhanVien(String hoTen, double luong){...}
    void xuat(){...}
    private double thueThuNhap(){...}
}
```

```
package poly.hcm;
public class TruongPhong extends NhanVien{
    public double trachNhiem;
    public TruongPhong(String hoTen, double luong, double trachNhiem){...}
    public void xuat(){
        // Mã ở đây có thể sử dụng những tài sản nào của lớp cha
    }
}
```

A. super.hoTen
B. super.luong
C. super.xuat()
D. super.thueThuNhap()

super keyword

- Từ khóa super dùng để truy xuất đến thành viên của lớp cha
- Cũng có thể gọi hàm khởi tạo của lớp cha thi hành thông qua từ khóa này

```
public class Parent{
    public String name;
    public void method(){}
}
```

```
public class Child extends Parent{
    public String name;
    public void method(){
        this.name = super.name;
        super.method()
    }
}
```

super keyword

```
package poly.ho;
public class NhanVien{
public NhanVien(String hoTen, double luong){...}
    public void xuat(){...}
}
```

```
package poly.hcm;
public class TruongPhong extends NhanVien{
    public double trachNhiem;
    public TruongPhong (String hoTen, double luong, double trachNhiem){
        super(hoTen, luong);
        this.trachNhiem = trachNhiem
    }
    public void xuat(){
        super.xuat()
        System.out.println(trachNhiem)
    }
}
```

Overridding

- Overridding là kỹ thuật cho phép định nghĩa lại 1 phương thức đã có ở lớp cha – super class, để sử dụng cho phù hợp với thiết kế của lớp con – sub class

```
public class Parent{
    public void method(){...}
}
```

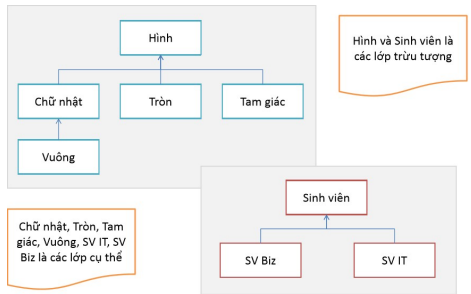
```
public class Child{
    public void method(){...}
}
```

Overridding

- Lớp con ghi đè phương thức của lớp cha thì sẽ che dấu phương thức của lớp cha
- Mục đích của ghi đè là để sửa lại phương thức của lớp cha trong lớp con
- Sử dụng từ khóa super để truy cập đến phương thức đã bị ghi đè của lớp cha.
- Đặc tả truy xuất của phương thức lớp con phải có độ công khai **bằng hoặc hơn** đặc tả truy xuất của phương thức lớp cha.

Abstract class :: Lớp trừu tượng

- Lớp trừu tượng là lớp có các hành vi chưa được xác định rõ
  - Ví dụ 1: Đã là hình thì chắc chắn là có diện tích và chu vi nhưng chưa xác định được cách tính mà phải là một hình cụ thể như chữ nhật, tròn, tam giác... mới có thể xác định cách tính
  - Ví dụ 2: Sinh viên thì chắc chắn có điểm trung bình nhưng chưa xác định được cách tính như thế nào mà phải là sinh viên của ngành nào mới biết được môn học và công thức tính điểm cụ thể.
- Vậy lớp hình và lớp sinh viên là các lớp trừu tượng vì phương thức tính chu vi, diện tích và tính điểm chưa thực hiện được.



- Từ khóa **abstract** được sử dụng để định nghĩa lớp và phương thức trừu tượng
- Phương thức trừu tượng là phương thức không có phần thân xử lý và được khai báo bằng từ khóa abstract.
- Lớp chứa phương thức trừu tượng thì lớp đó phải là lớp trừu tượng.
- Trong lớp trừu tượng có thể định nghĩa các phương thức cụ thể hoặc khai báo các trường
- Không thể sử dụng new để tạo đối tượng từ lớp trừu tượng.

```
abstract public class MyClass{
    abstract public type MyMethod();
}
```

Sử dụng từ khóa abstract để định nghĩa lớp và phương thức trừu tượng

```
abstract public class SinhVien{
    abstract public double getDiemTB();
}
```

```
abstract public class Hinh{
    abstract public double getChuVi();
    abstract public double getDienTich();
}
```

```
abstract public class SinhVien{
    public String hoTen;
    abstract public double getDiemTB();
}
```

```
public class SinhVienIT extends SinhVien{
    public double diemJava;
    public double diemCss;
    @Override
    public double getDiemTB(){
        return (2 * diemJava + diemCss)/3;
    }
}
```

```
public class SinhVienBiz extends SinhVien {
    public double keToan;
    public double marketting;
    public double banHang;
    @Override
    public double getDiemTB(){
        return
        (keToan + marketting + banHang)/3;
    }
}
```

Polymorphism :: Tính đa hình

- Overriding thực hiện tính đa hình trong lập trình hướng đối tượng (một hành vi được thể hiện với các hình thái khác nhau)
- Gọi phương thức bị ghi đè được quyết định lúc chạy chương trình (runtime) chứ không phải lúc biên dịch chương trình (compile time)

```
abstract public class DongVat{
    abstract public void speak();
}
```

```
public class Cho extends DongVat{
    public void speak(){
        System.out.println("Woof");
    }
}
```

```
public class Meo extends DongVat{
    public void speak(){
        System.out.println("Meo");
    }
}
```

```
public class Vit extends DongVat{
    public void speak(){
        System.out.println("Quack");
    }
}
```

```
DongVat cho = new Cho();
DongVat meo= new Meo();
DongVat vit = new Vit();

cho.speak();
meo.speak();
vit.speak();
```

Nhớ gì ?!!!

- Tại sao phải cần đến thừa kế class ?!

Nhớ gì ?!!!

- Kế thừa trong Java là gì? Kế thừa là một tính năng quan trọng trong Java cho phép các đối tượng được xây dựng dựa trên các đối tượng khác. Kế thừa cho phép các đối tượng con kế thừa các thuộc tính và phương thức của đối tượng cha. Kế thừa được thực hiện bằng từ khóa "extends".

Nhớ gì ?!!!

- Khi nào bạn nên sử dụng kế thừa trong Java? Bạn nên sử dụng kế thừa trong Java khi có một số đối tượng trong chương trình của bạn chia sẻ các thuộc tính hoặc phương thức giống nhau. Thay vì lặp lại mã lệnh để tạo ra các đối tượng mới, bạn có thể tạo một lớp cha và cho các lớp con kế thừa các thuộc tính và phương thức của nó.

Nhớ gì ?!!!

- Lớp cha và lớp con trong Java là gì? Lớp cha trong Java là lớp mà các lớp con được kế thừa từ đó. Lớp con là lớp được tạo ra bằng cách kế thừa từ lớp cha và có thể chứa các thuộc tính và phương thức riêng của nó.

Nhớ gì ?!!!

- Từ khóa "extends" trong Java có nghĩa là gì? Từ khóa "extends" trong Java được sử dụng để kế thừa một lớp cha. Ví dụ, nếu bạn muốn tạo một lớp con từ một lớp cha, bạn sẽ sử dụng từ khóa "extends" sau đó là tên của lớp cha, như sau:

```
class ChildClass extends ParentClass {
    // code here
}
```

Nhớ gì ?!!!

- Lớp con có thể kế thừa nhiều lớp cha không? Không, trong Java, một lớp con chỉ có thể kế thừa từ một lớp cha duy nhất. Điều này được gọi là "đơn kế thừa".

Nhớ gì ?!!!

- Lớp cha có thể kế thừa từ lớp con không? Không, trong Java, một lớp cha không thể kế thừa từ một lớp con.

Nhớ gì ?!!!

- Phương thức "super()" trong Java được sử dụng để làm gì? Phương thức "super()" trong Java được sử dụng để gọi một phương thức của lớp cha. Nó được sử dụng để gọi phương thức của lớp cha khi lớp con có một phương thức trùng tên với phương thức của lớp cha.

Bài tập

- Thiết kế một chương trình quản lý nhân viên trong công ty. Có hai loại nhân viên: Nhân viên bán hàng và Nhân viên kế toán. Cả hai loại nhân viên đều có các thông tin chung như tên, tuổi, giới tính, địa chỉ, số điện thoại:
  - Nhân viên bán hàng có thêm thông tin số giờ làm việc và ***doanh số bán hàng***, lương tính bằng nhân số giờ làm việc với mức lương cơ bản + 10%doanh số
  - Nhân viên kế toán có thêm thông tin số giờ làm việc và ***lương cơ bản***, lương tính bằng nhân số giờ làm việc với lương cơ bản
- Thiết kế các lớp để xây dựng chương trình quản lý này gồm các chức năng: nhập, xuất, tính lương

Hướng dẫn

Lớp cha: NhanVien (Nhân viên)

- Thuộc tính: tên, tuổi, giới tính, địa chỉ, số điện thoại, số giờ làm việc
- Phương thức:
  - Khởi tạo (constructor): truyền vào tên, tuổi, giới tính, địa chỉ, số điện thoại, số giờ làm việc
  - Tính lương (abstract): trả về lương của nhân viên
  - In kết quả: in thông tin của nhân viên

Hướng dẫn

Lớp con 1: NhanVienBanHang (Nhân viên bán hàng)

- Thuộc tính: số doanh số bán hàng, lương cơ bản
- Phương thức:
  - Khởi tạo (constructor): truyền vào tên, tuổi, giới tính, địa chỉ, số điện thoại, số giờ làm việc, ***số doanh số bán hàng***, lương cơ bản
  - Tính lương (override): trả về lương của nhân viên bán hàng, được tính bằng cách nhân số giờ làm việc với mức lương cơ bản (ví dụ: 100,000 đồng/giờ) cộng với 10% tổng doanh số bán hàng.
  - In kết quả (override): in thông tin nhân viên bán hàng

Hướng dẫn

Lớp con 2: NhanVienKeToan (Nhân viên kế toán)

- Thuộc tính: lương cơ bản
- Phương thức:
  - Khởi tạo (constructor): truyền vào tên, tuổi, giới tính, địa chỉ, số điện thoại, số giờ làm việc, lương cơ bản
  - Phương thức tính lương (override): trả về lương của nhân viên kế toán, được tính bằng cách nhân số giờ làm việc với mức lương cơ bản.
  - In kết quả (override): in thông tin nhân viên kế toán

Bài tập

- Hãy giả định rằng bạn đang viết một ứng dụng quản lý sinh viên cho trường đại học của bạn. Mỗi sinh viên có các thuộc tính như sau: tên, mã số sinh viên, ngày tháng năm sinh, giới tính, điểm trung bình, và lớp học.
- Bạn hãy tạo ra các lớp để mô tả các đối tượng sau:
  - Lớp cha Student (sinh viên) với các thuộc tính chung của tất cả sinh viên. Lớp này có các phương thức in thông tin sinh viên
  - Lớp con UndergraduateStudent (sinh viên đại học) kế thừa từ lớp cha Student và có thêm thuộc tính về chuyên ngành của sinh viên. Lớp này override phương thức in thông tin và in thông tin sinh viên đại học
  - Lớp con GraduateStudent (sinh viên sau đại học) kế thừa từ lớp cha Student và có thêm thuộc tính về loại hình học vụ của sinh viên (nghiên cứu hoặc chuyên ngành). Lớp này override phương thức in thông tin và in thông tin sinh viên sau đại học

Trường Đại học Nguyễn Tất Thành
Khoa Công Nghệ Thông Tin

-Hết chương 4-